

Stage 2 Development Brief

Recovering Structure From Damaged or Poorly Scanned Documents

Field	Detail
Problem ID	ML-T2-022
Candidate	Gyimah Ramsey Opoku
Student ID	9021623
Stage	Stage 2 — Implementation Planning & Environment Setup
Status	Stage 1 Research Brief submitted and confirmed; no Stage 2 guide issued, plan below is self-directed
Duration	Following Stage 1 (2–3 week track)

1. Purpose of This Stage

Stage 1 established the research foundation: problem formulation, literature review, data strategy, and four candidate technical approaches (A–D). This document records the development plan for Stage 2, translating that research into a concrete, staged implementation, before any code is treated as final. As with Stage 1, decisions here are made deliberately and are expected to be defensible at review, not just functional.

2. Development Environment & Stack

No fixed stack was provided by the internship track; all choices below are made independently within the zero-cost constraint carried over from Stage 1.

Component	Environment	Reason
OCR baseline, data prep, mask generation	Local machine	CPU-sufficient; no GPU required for these stages, keeps iteration fast
MLM fine-tuning (Approach B)	Kaggle Notebooks	Free weekly GPU quota (30 hrs), more predictable than Colab's free-tier limits for a multi-session fine-tuning workflow
OCR engine	Tesseract (via pytesseract)	Free, widely documented, directly comparable against CER/WER figures reported in reviewed literature
Image handling	OpenCV, Pillow	Standard tooling for preprocessing and mask application
Text reconstruction	HuggingFace transformers	Matches the masked-language-model

Component	Environment	Reason
	(RoBERTa-base)	approach used in DocRevive (Stage 1 literature review); pretrained weights available at zero cost
Mask generation	Custom implementation (Yu et al. free-form algorithm)	No existing library targets document-specific irregular tears; written in-house as identified in Stage 1 as a genuine gap
Evaluation	jiwer (CER/WER), custom ECE implementation	jiwer is the standard lightweight library for edit-distance metrics; ECE has no single standard library and is implemented directly

3. Build Philosophy: Staged, Not End-to-End

Rather than building one integrated pipeline from the start, each component is implemented and independently validated before integration. This is slower to reach a demonstrable end-to-end result, but produces a defensible chain of evidence: at review, progress can be shown as measured CER/WER/ECE at each stage rather than a single black-box result. This also isolates failures to a specific component rather than an entangled pipeline, which matters given several sub-tasks (reconstruction, uncertainty tagging) are experimental rather than off-the-shelf.

4. Stage-by-Stage Implementation Plan

Step	What is built	Environment	Validated by
1. Baseline OCR	Plain OCR (Tesseract) run on DIBCO/H-DIBCO images, no restoration applied	Local	CER/WER against DIBCO ground truth; this is the reference every later step must beat
2. Synthetic torn-region data	Free-form irregular mask generator applied to clean/DIBCO images to simulate torn or missing regions	Local	Visual inspection of mask realism; confirms masks resemble irregular tears, not rectangles
3. MLM-based reconstruction	RoBERTa fine-tuned to predict missing spans/words using surrounding context (Approach B)	Kaggle	CER/WER on reconstructed spans vs. ground truth, compared against Step 1 baseline
4. Uncertainty-tagging layer	Confidence scoring layered on top of Step 3's output; low-confidence spans flagged as uncertain (Approach C)	Kaggle / Local	Expected Calibration Error (ECE); checks whether confidence scores are trustworthy, not just present
5. Integration	Stitch Steps 1–4 into a single pipeline accepting a degraded page image and producing the structured JSON output defined in Stage 1	Local	End-to-end run on held-out samples from both degradation types (faded text, torn regions)

Approach D (GAN/diffusion-based visual restoration) remains a stretch extension after Step 5, only if time allows, as scoped in Stage 1.

5. Success Criteria per Stage

- Step 1: Baseline CER/WER recorded and matches the general range reported in reviewed literature for uncorrected OCR on degraded text.
- Step 2: Generated masks are visually distinct from rectangular occlusions and approximate irregular tear boundaries.
- Step 3: Reconstruction reduces CER/WER on masked spans relative to leaving them as OCR errors or blanks.
- Step 4: ECE is measured and reported honestly, including if it underperforms the ~0.685 F1 reference from Stage 1's literature review; this is a valid and expected finding, not a failure to hide.
- Step 5: Full pipeline runs end-to-end on both degradation types without manual intervention, producing valid structured JSON output.

6. Risks and Open Questions Carried Forward From Stage 1

- Whether MLM reconstruction trained on rectangular occlusions generalizes to irregular tear-shaped masks, or requires retraining specifically on tear-shaped data (to be tested at Step 3).
- Whether synthetic torn-region masks are a realistic enough proxy for physical damage (to be assessed visually at Step 2 and revisited if Step 3/4 results look inconsistent with expectations).
- How to aggregate confidence across multiple task heads if layout/reading-order prediction is added after Step 5.

7. Next Steps

Implementation begins at Step 1 (baseline OCR). Each step is completed and its validation criteria checked before moving to the next, per the build philosophy in Section 3. Progress and any deviations from this plan will be recorded as they occur, rather than reconstructed retroactively at the next review.